

Reading Seoul Through Its Movement

A 50-Minute Guest Lecture Design Document

Course: Data Visualization and Business Analytics (MSc)

0. Document Purpose

This is the **content & narrative design document** for a 50-minute guest lecture (+ 10-min Q&A) on data visualization applied to real research on Seoul urban data.

This document delivers:

- 1. Full presentation narrative arc, section-by-section
- 2. Every visualization the presenter will show, with the *reason* it must be that specific chart type (this is a Data Viz class)
- 3. Data sources, preprocessing logic, and analytical method for each visualization
- 4. Hooks to code examples that will be built separately in Jupyter
- 5. Design specs for the accompanying interactive GitHub Pages deployment

This document does NOT yet contain:

- Final slide layouts (PPT step is downstream)
- Complete Python code (comes after this design is approved)
- Interactive deployment code (comes after Jupyter code is validated)

Language rule: All presentation-facing content, including every chart title, axis label, legend, and annotation, must be in English. The MD file itself uses English for all deliverable-facing text; Korean is used only in `[NOTE: ]` blocks for the presenter's private reference.

1. Meta

Field	Value
Total speaking time	50 minutes + 10 minutes Q&A
Audience	Master's-level students, weak coding & finance background
Course context	Data Visualization and Business Analytics

Field	Value
Deliverable format	Slide deck (English) + live-ish code demos + interactive web page
Presenter framing	Working professional (backend/DB 26 months + development PM in fintech and video). No mention of undergraduate status.
Research scope framing	"I am analyzing all 25 districts of Seoul. Given today's time, I will focus on 3 representative commercial hubs and share what I found. The full 25-district version is ongoing."

2. The Big Idea (One Sentence)

The way Seoul moves reveals what Seoul is. If you understand the flow, you understand where to live, work, invest, and build.

This one sentence is repeated visually at the start, middle, and end of the talk. It gives the audience a coat hook to hang every visualization on.

3. Narrative Arc (Why This Order)

The talk is built as a **question → data → surprise → deeper question → answer** loop, which is standard for good data storytelling. The audience is not fluent in coding or finance, so every technical concept enters through a lived experience they already have (riding the subway, choosing where to live).

- Hook (a decision everyone faces)
- Who am I & why I study this
 - Frame the research question
 - Show the data (make the invisible visible)
 - Test a common assumption (surprise moment)
 - Go deeper: find hidden structure
 - Show how AI helped me build all of this
 - Live interactive demo
 - What this means for you
 - Q&A

4. Section-by-Section Design

Each section below specifies:

- Time budget
 - What the audience sees
 - What the presenter says (message, not verbatim script)
 - Every visualization used, with the *reason* it is that type
 - Data + preprocessing behind it
 - Code snippet reference (built in Jupyter later)
-

SECTION 1 — Hook (0:00 - 2:30, 2.5 min)

Slide 1.1 — Title slide

- Title: **"Reading Seoul Through Its Movement"**
- Subtitle: *"A data story about where 10 million people go every day, and what it tells us about where to live."*
- Presenter name, date, "Guest Lecture · Data Visualization and Business Analytics"

Slide 1.2 — The Question

- Full-bleed photograph background (Seoul at night from a high angle) with a single English line overlaid:
- *"Your friend calls tonight and says: 'I'm moving to Seoul next month. Where should I live?' What do you tell them?"*
- **No chart on this slide.** The absence of data is deliberate — it primes the audience to want data.

Message: Everyone has been on both sides of this question. Today we will answer it not with opinion, but with about 200 million subway taps.

[NOTE: 이 쪽에서 학생들이 자기 얘기라고 느끼게 만드는 것이 핵심. 통계 숫자는 여기서 하나만 강하게 던짐: "200 million subway taps"]

SECTION 2 — About the Presenter (2:30 - 5:30, 3 min)

Slide 2.1 — Career one-pager

- Left half: portrait / avatar / logo
- Right half: three horizontal bands (timeline style)
 - **Band 1 · 26 months** — Backend & Database Engineer (military alternative service, IT industry)
 - **Band 2 · 6+ months (ongoing)** — Development Project Manager
 - **Band 3 · Current focus** — Fintech development & video-platform development

- Bottom line: *"I ship real systems. That's how I learned that data without a decision behind it is decoration."*

Visualization: a simple horizontal timeline bar (built in matplotlib as demo #0 later — trivial code, but sets tone that even the presenter's bio is data).

Why this chart type: Timeline horizontal bar is the most information-dense way to show duration + parallel roles + present-vs-past in a single glance. A pie chart would be wrong here (no part-to-whole meaning); a table would be too dry.

[NOTE: 학부생이라는 말 절대 금지. "working professional in fintech and video platform development" 이라고 자기 소개.])

SECTION 3 — Framing the Research Question (5:30 - 8:30, 3 min)

Slide 3.1 — From personal question to research question

Show three progressively formalized versions of the same question:

1. *"Where should I live in Seoul?"* (everyday)
2. *"Which Seoul neighborhood matches which kind of life?"* (product-manager voice)
3. *"Can subway ridership patterns act as a fingerprint that classifies Seoul neighborhoods by function, and can that classification predict quality-of-life factors relevant to a residence decision?"* (research voice)

Slide 3.2 — Scope disclosure

Bullet points, English:

- Full study covers all 25 districts of Seoul (in progress).
- Today's talk focuses on **three representative hubs** chosen for maximum contrast:
 - **Gangnam** — corporate & commerce
 - **Hongdae (Hongik University area)** — youth & nightlife
 - **Yeouido** — pure business & finance
- Data window: **most recent full 24 months** of subway tap-in / tap-out data + latest available apartment price and demographic data.

Slide 3.3 — Hypotheses (state them up front)

Present three testable hypotheses. Stating them before analysis is a discipline this audience will respect.

- **H1:** *"All busy subway stations are basically the same."* — The Null Hypothesis of Naïve Intuition. Expected outcome: rejected.
- **H2:** Hourly ridership curves cluster into a small number (~4-6) of distinct "neighborhood types" that generalize across the city.

- **H3:** The dominant ridership pattern of a neighborhood predicts apartment price movement direction over the study window, controlling for baseline price level.

[NOTE: H1을 재미있게 반박하는 것이 이 발표의 "surprise moment". 학생들이 "다 비슷할 것 같은 데?" 라고 생각하다가 데이터로 뒤통수 맞음.]

Visualization on Slide 3.3: a simple **hypothesis card matrix** — 3 cards side by side, each with H#, statement, expected outcome, and how it will be tested. This is not a chart but a designed visual asset (will be built as clean HTML/CSS in the interactive site and as a matching PPT slide).

SECTION 4 — Data Journey: Making the Invisible Visible (8:30 - 18:30, 10 min)

Goal of this section: earn the audience's trust by showing you actually did the boring work, and use each visualization to teach one visualization principle.

4.1 Where the data comes from

Slide 4.1 — Data source panel

A single slide with 4 boxes, each showing dataset name, URL/API, size, and update frequency:

1. **Subway ridership by station × hour** — Seoul Open Data Plaza (OA-12252). Monthly release, ~24 months collected. ≈ 5-10 M rows.
2. **Subway ridership by station × day** — Seoul Open Data Plaza (OA-12914). Daily release.
3. **Apartment real-transaction prices** — Ministry of Land, Infrastructure and Transport (MOLIT) real-transaction API. Monthly.
4. **Station coordinates & administrative boundary GeoJSON** — Seoul Open Data Plaza + national statistical geography portal.

[NOTE (팩트체크 결과):] (- 서울 열린데이터광장 생활이동 OD는 현행화 중지 및 개편 중이므로 사용하지 않음.) (- 대신 지하철 승하차 데이터로 flow를 재구성. 이것이 오히려 더 오래된 시계열을 안정적으로 확보 가능.) (- OA-12252는 매월 5일 전월 데이터 갱신, OA-12914는 매일 3일 전 데이터 갱신.)

Visualization used: a **data lineage diagram** — a horizontal flow chart with icons: [Raw APIs] → [Landing files] → [Cleaned tables] → [Analysis-ready features] → [Charts].

Why this chart type: A lineage diagram is the correct visualization for "here is how bytes became insight." A funnel chart would misrepresent the shape (there is no drop-off metric). A Sankey is overkill because the flow is deterministic, not proportional.

4.2 Preprocessing — the honest slide

Slide 4.2 — What "clean data" actually means

Show a **before/after side-by-side** of a real messy row from OA-12252:

Left (raw): Korean column names mixed with English, thousands separators as strings, station names with parenthetical alternate names, redundant total row at the bottom of each day.

Right (clean): normalized snake_case English column names, integer types, station names canonicalized, a `(station_id)` foreign key, `(datetime)` column.

Visualization used: a two-column diff-style table — annotated with red highlights on what was wrong and green on what was fixed.

Why this specific choice: the audience is weak at coding. Showing that data preprocessing is a *readable, visible* transformation demystifies it. Charts are inappropriate here — text-with-highlighting is the right medium.

Code snippet reference: `(01_preprocessing.ipynb)` — will be shown briefly (~15 seconds).

[NOTE: 학생들이 "아 전처리라는 게 저런 거구나" 하고 확실히 이해하는 순간. 코드보다 결과의 시각적 대비가 중요.]

4.3 The first real visualization — ridership landscape

Slide 4.3 — "This is Seoul, breathing."

Chart A — Hourly ridership heatmap of the three focus stations

- **Type:** heatmap, x-axis = hour of day (0-23), y-axis = day of week (Mon-Sun), color intensity = mean tap-in count.
- **Layout:** three heatmaps side by side, one per station, shared color scale.
- **Library:** seaborn `(heatmap)` on a pivot table.
- **Why this chart type:**
 - Ridership varies on two cyclical axes (hour, weekday). A heatmap is the *only* chart that shows both cycles with density at each cell.
 - A line chart would hide the weekend pattern; a bar chart would fragment the picture.
 - Sharing the color scale across the three panels is critical so the audience can compare *magnitudes*, not just shapes.
- **What the audience will see immediately:** Gangnam and Yeouido "burn" during weekday rush hours and go quiet on weekends. Hongdae is the opposite — it lights up at night and on weekends. This is the first "aha" moment.

Message: "You just watched three neighborhoods reveal their soul, without a single label."

Chart B — Multi-line hourly ridership curves

- **Type:** multi-line chart, x-axis = hour of day, y-axis = mean tap-in, one colored line per station.

- **Library:** matplotlib.
- **Why this chart type after the heatmap?** Because now the audience wants the shape isolated. A line chart is precisely designed for "how does one continuous quantity move against another continuous quantity." Overlaying three lines lets us contrast the *shape* of each station's pulse directly.
- **Design detail:** annotate each line's peak with a small callout label — this is a Data Viz teaching moment: annotations turn a chart into an argument.

[NOTE: 여기서 학생들에게 "왜 heatmap 다음에 line chart를 또 보여줄까?" 라고 물어보고 답 유도. Data Viz 수업이니까 chart 선택 이유를 계속 강조.]

SECTION 5 — Testing the Assumption: The Surprise (18:30 - 30:30, 12 min)

This is the argumentative core of the talk. We take the audience's naïve intuition (H1) and dismantle it with data.

5.1 Setting up H1

Slide 5.1 — "Aren't they all just busy stations?"

Show a **grouped bar chart** of total annual ridership at the three stations. All three look enormous (tens of millions).

Chart C — Grouped bar chart, annual ridership totals

- **Type:** vertical grouped bars, x = station, y = total taps, color = year.
- **Why this chart type:** A bar chart is the standard for comparing discrete categories on a single metric. This is the *setup* for the punchline: on this axis they look nearly identical. Deliberately unhelpful is the point.
- **Rhetorical setup:** *"On this chart, they all look the same. This is what H1 predicts. But let's zoom in on when the taps happen."*

5.2 Rejecting H1

Chart D — Weekday vs weekend ridership ratio (diverging bar)

- **Type:** diverging horizontal bar chart, centered at 0. Positive direction = weekend-heavy, negative direction = weekday-heavy.
- One bar per station.
- **Why this chart type:** A diverging bar chart is the correct visualization when one metric can go in two meaningfully opposite directions. It makes the *direction* of a neighborhood's personality instantly visible.
- **Immediate reading:** Gangnam and Yeouido plunge deep into "weekday-heavy" territory. Hongdae leans strongly weekend-heavy. **H1 is dead.**

Chart E — Radar (spider) chart of neighborhood profile

- **Type:** radar chart with 6 axes:
 1. Morning peak intensity
 2. Evening peak intensity
 3. Weekend/weekday ratio
 4. Late-night ridership share
 5. Directional asymmetry (tap-in vs tap-out at peak)
 6. Ridership volatility (coefficient of variation)
- Three overlaid polygons, one per station, semi-transparent.
- **Library:** `plotly` (`Scatterpolar`) or `matplotlib` polar projection.
- **Why this chart type:**
 - Radar charts are *often criticized* in data viz. This is a teaching moment: we use it correctly here because we have (a) a fixed small number of axes, (b) axes with directly comparable normalized scales, (c) a purpose of showing *shape/profile* rather than precise values. This makes it defensible.
 - Explicitly acknowledge to the audience that radar charts are controversial, then explain why this use-case is one of the exceptions. This makes the presenter look like they know the field.

[NOTE: 여기가 발표의 Data Viz 관점 하이라이트 중 하나. 학생들에게 "radar chart는 원래 나쁜 시각화로 자주 비판받지만, 이런 조건에서는 정당하다"고 설명하면 교수님이 좋아하실 것.]

5.3 Statistical grounding

Slide 5.3 — Are these differences statistically real?

Introduce a one-slide, minimally technical explanation of a Kolmogorov-Smirnov test comparing the hourly distributions of any two stations.

Chart F — Empirical CDF (cumulative distribution) plot with KS distance annotated

- **Type:** step-line CDF, one line per station, with a vertical arrow marking the maximum vertical distance between two CDFs (which is the KS statistic).
- **Library:** `matplotlib`.
- **Why this chart type:** The CDF is the geometric object the KS test operates on. Showing the test as a *picture* — a vertical distance between curves — makes an inferential statistic legible to non-statisticians. This is a rare and powerful visualization in a Data Viz class.
- **Message:** the shapes are not just visually different, they are *distributionally* different, with $p < 0.001$.

[NOTE: KS test 결과 p-value 하나만 크게 보여주고 "이건 우연이 아니다"라고 강조. 통계 배경 없어도 이해 가능.]

5.4 Directional asymmetry — the tap-in vs tap-out signature

Chart G — Butterfly / back-to-back bar chart at peak hour

- **Type:** horizontal bars mirrored around a central axis: left side = tap-out volume at 08:00–09:00, right side = tap-in volume at 08:00–09:00, one row per station.
 - **Why this chart type:** Butterfly bar charts are designed precisely for two-directional comparisons of the same subjects. Yeouido will show huge tap-out (people arriving to work) with tiny tap-in in the morning. Hongdae will be nearly symmetric. This chart *proves* the "purpose" of each neighborhood in a single image.
-

SECTION 6 — Deeper Patterns: Hidden Structure (30:30 - 40:30, 10 min)

Now we go from three stations to the whole city, then zoom back in with new insight.

6.1 From 3 stations to 300+ stations

Slide 6.1 — Scaling up: "If three stations tell us this much, what does the whole subway system say?"

Motivate H2: if we take the 24-hour ridership curve of every station in Seoul and let a clustering algorithm group them, do we find natural "neighborhood types"?

6.2 Feature engineering (briefly)

Chart H — Feature table with intuitive icons

- Not a data chart. It's a designed slide showing the 6 hand-crafted features used to profile each station (same as radar axes but formalized as a table).
- **Why not a chart:** because this is definitional content. Forcing it into a chart would be misuse.

[NOTE: 이 슬라이드는 코드에서 실제로 만들어지는 `feature engineering`을 보여줌. Jupyter에서 `df.describe()` 결과를 예쁘게 정리한 이미지도 하나 첨부.]

6.3 The clustering

Chart I — 2D UMAP scatter, colored by KMeans cluster

- **Type:** UMAP dimensionality reduction of the 6-feature space to 2D. Each dot is one station. Color = KMeans cluster (k chosen by elbow/silhouette).
- **Library:** umap-learn + scikit-learn KMeans + plotly for interactivity, matplotlib for the static export.
- **Why this chart type:**
 - We have high-dimensional feature space; we cannot plot 6 axes.
 - UMAP preserves local structure better than PCA for cluster visualization, which matters when the goal is "do points fall into visually separated groups."

- Coloring by cluster label lets the audience see whether the algorithm's groupings look natural.
- **Interactive layer (for the web version):** hovering a point reveals the station name and its 6 feature values.

Chart J — Cluster centroid radar chart

- **Type:** radar chart, one polygon per cluster centroid.
- Same axes as Chart E, but now the polygons are cluster archetypes, not individual stations.
- **Why this chart type:** we already used radar for individual stations; now we reuse it for aggregate cluster identity. Consistency of chart choice reinforces the audience's mental model. This is itself a Data Viz principle worth naming: **visual language consistency across a talk**.

Message: The algorithm found ~5 recognizable neighborhood types without ever being told what a neighborhood is. Types include: Business District, Nightlife District, Residential Commuter, Mixed Residential-Retail, Transit Interchange.

[NOTE: k=5가 나오면 이상적. 실제로는 데이터 봐야 알겠지만 대략 그 정도. Jupyter에서 elbow curve와 silhouette score로 k 결정하는 과정도 슬라이드에 짧게 넣음.]

6.4 Zoom back to the three focus stations

Slide 6.4 — "Where did our three stations land?"

Recolor the UMAP scatter to highlight only the three focus stations, plotted against faded cluster background.

Chart K — Annotated UMAP with focus stations highlighted

- **Type:** same UMAP as Chart I, but with the three focus stations enlarged, labeled, and surrounded by faded cluster clouds.
- **Why this chart type:** this is a **layered visualization** — background context (all stations) + focused subject (the three we care about). Layering is a core data viz principle when moving between the general and the specific.

Chart L — Chord diagram of transitions between top-flow station pairs

- **Type:** chord diagram from the `holoviews`/`bokeh` `Chord` element, showing that morning taps-in at Gangnam largely originate as taps-out from a specific set of residential clusters, and similarly for the other two focus areas.
- **Data preprocessing needed:** since true OD data is unavailable during the current data-portal renovation, we approximate flow by matching morning tap-outs at residential-cluster stations to morning tap-ins at business-cluster stations by hourly volume correlation. This limitation is disclosed on-slide (see 6.5).

- **Why this chart type:** chord diagrams are designed for showing pairwise flows in a bounded set of nodes. Sankey would be an alternative but reads better for strictly directional flow; chord reads better for "these places are linked to those places."

6.5 Honest limitations

Slide 6.5 — What this analysis can NOT do

Bullet list, on-screen, in the presenter's own voice:

- Approximate flow is not real origin-destination.
- Public data lags real-time by weeks.
- Tap-in/tap-out counts unique cards, not unique humans.
- Correlation between ridership pattern and apartment price movement is *not* causation.

Why this slide exists: Every senior researcher discloses limitations. Doing this in a student-facing talk models good practice and preempts the sharpest Q&A questions.

[NOTE: 이 슬라이드가 있냐 없냐로 발표 수준이 갈림. 반드시 포함.]

6.6 Connecting ridership pattern to apartment price movement (H3)

Chart M — Small multiples: per-cluster monthly apartment price index

- **Type:** small multiples (a.k.a. faceted / trellis) line chart. 5 mini panels (one per cluster), each showing the mean monthly apartment price index of stations in that cluster over the 24-month window. Y-axis shared for comparability.
- **Library:** matplotlib subplots or seaborn `FacetGrid`.
- **Why this chart type:** small multiples are the correct choice when comparing the *shape* of the same variable across multiple categories. Overlaying five lines in one plot would tangle. Small multiples separate them cleanly while retaining direct comparability via shared axes.
- **This is a Data Viz teaching moment:** state explicitly that Edward Tufte's *small multiples* principle is what we are using.

Chart N — Slope chart: price change from month 0 to month 24, by cluster

- **Type:** slope chart (a.k.a. dumbbell chart variant). Left endpoint = mean price at start, right endpoint = mean price at end, one line per cluster, sorted by change magnitude.
- **Why this chart type:** slope charts are designed exactly for "before → after" comparisons across many categories. They collapse a 24-month time series into a single sharp comparison.

Message: The commuter-residential cluster and the business-district cluster diverge in price trajectory. This partially supports H3.

SECTION 7 — AI as a Co-pilot: How I Actually Built This (40:30 – 45:30, 5 min)

Professor's explicit request: show students how the presenter uses AI *practically* to build code and projects.

Slide 7.1 — The three ways I use AI on this project

Format: three columns, each with a title and 2–3 concrete examples.

- **Column 1 · Boilerplate acceleration** — data-loading skeletons, API request patterns, dataframe reshaping code. Concrete: "Here's the Seoul Open Data API response schema, write me a paginated loader." Result: 30 minutes saved every dataset.
- **Column 2 · Chart iteration** — "Here is the dataframe. Give me a chart that shows X and Y simultaneously." Then I iterate: color palette, annotation, axis formatting. Result: I focus on the *idea* of the chart, AI handles the syntax.
- **Column 3 · Debugging & explanation** — Paste an error, paste a stubborn merge, ask for a diagnosis. And equally valuable: "Explain to me why my KS test result looks like this."

Slide 7.2 — What AI does NOT do for me

Explicit list:

- AI does not choose the research question.
- AI does not decide which chart is honest.
- AI does not know which limitation to disclose.
- AI does not know my audience.

Message: *"The best analogy I have: AI is a very fast junior developer who has read every library manual. I am still the one who has to say what the project is for."*

[NOTE: 이 섹션이 교수님이 명시적으로 요청한 부분. 학생들이 가장 궁금해하는 내용이기도 함. 과정 없이, 실제 사용법 그대로 이야기하는 것이 중요.]

Slide 7.3 — Live snippet demo (~90 seconds)

Open a Colab tab (or embedded gif animation on slide) and show one real interaction: paste a small dataframe, ask for a specific chart, get code, run it, chart appears.

Visualization used: the chart that pops out is a **stacked area chart** of morning tap-out distribution by residential cluster, feeding into the top-3 business stations. This is a chart we have not yet seen in the talk — so the AI demo produces new information, not a repeat.

Chart O — Stacked area chart (produced live via AI-assisted code)

- **Type:** stacked area chart, x = hour of day, y = proportional flow, colored bands by originating residential cluster.
- **Why this chart type:** stacked area is correct when the parts-of-whole change *continuously* over a continuous x-axis. It shows both the total and the composition

simultaneously.

SECTION 8 — Live Interactive Demo (45:30 - 48:30, 3 min)

Open the GitHub Pages URL live. Show the interactive prototype:

- Left pane: raw data preview (a live pandas DataFrame excerpt as a table).
- Middle: a row of clickable transformation buttons (representing library functions like `resample`, `groupby`, `pivot_table`, `merge`, `StandardScaler`, `KMeans.fit`).
- Right pane: the resulting visualization that updates as the user clicks each transformation.

Design goal: the audience can literally see the pipeline from raw → chart animate, one step at a time. Each step has a tooltip explaining the library, function, and *why this step matters for the final chart to be honest and readable*.

Chart P — The interactive is itself the chart. Under the hood it will be a Plotly Dash app deployed as a static export (using Dash's static rendering) OR a Plotly + vanilla JS page hosted on GitHub Pages. Final tech choice will be validated in the deployment step.

[NOTE: 이 부분이 GitHub Pages 배포의 핵심. 사용자 요청: "raw data 좌측, 라이브러리 기능 화살표 중간, 시각화 결과 우측, 클릭하면 동적으로 움직임." 이 요구사항 정확히 구현.]

SECTION 9 — Wrap-up (48:30 - 50:00, 1.5 min)

Slide 9.1 — What we did today

A recap slide showing the same three-hypothesis card matrix from Section 3, now with each card marked "Rejected" / "Supported" / "Partially supported" and a small thumbnail of the key chart that decided each hypothesis.

Slide 9.2 — What comes next in this project

- 25-district full rollout
- Add bus + bike + taxi flows to triangulate movement
- Predictive model: given a neighborhood's ridership signature, predict 12-month apartment price direction
- Public-facing tool: type your commute needs and lifestyle, get a ranked list of neighborhoods

Slide 9.3 — What I hope you take away

Three lines, big text:

1. Choose the chart before writing the code.

2. **Every chart is an argument. Make it honest.**

3. **AI is a lever, not the arm.**

Slide 9.4 — Q&A + resources

- QR code to the live interactive site (GitHub Pages URL).
 - QR code to the GitHub repo (2026-Summer-Study).
 - Contact.
-

5. Complete Visualization Inventory

For quick reference and later PPT/code planning, every chart with its purpose:

#	Chart	Type	Library	Section	Purpose
Bio	Career timeline	Horizontal timeline bar	matplotlib	2	Set analytical tone from the bio
A	Hourly $\times$ weekday ridership	Heatmap $\times 3$ panels	seaborn	4.3	First "aha", show neighborhoods have personality
B	Hourly ridership curves	Multi-line	matplotlib	4.3	Isolate and compare shapes
C	Annual totals	Grouped bar	matplotlib	5.1	Setup for H1 rejection
D	Weekday/weekend ratio	Diverging bar	matplotlib	5.2	Reveal directional personality
E	Station profile	Radar (spider)	plotly	5.2	Multi-dimensional identity in one glance
F	Ridership CDF	Step-line ECDF w/ KS distance	matplotlib	5.3	Make statistical test visible
G	Tap-in vs tap-out at peak	Butterfly bar	matplotlib	5.4	Prove neighborhood purpose
H	6-feature table	Designed table	(slide asset)	6.2	Definition slide, not a chart
I	Station clustering	UMAP scatter, cluster colored	umap + plotly	6.3	Reveal natural neighborhood types
J	Cluster centroids	Radar	plotly	6.3	Aggregate archetypes
K	Focus stations located	Layered UMAP scatter	plotly	6.4	Zoom back to specifics
L	Approximate morning flow	Chord diagram	holoviews/bokeh	6.4	Show inter-neighborhood linkage
M	Per-cluster price series	Small multiples line	seaborn FacetGrid	6.6	Compare shapes of price movement

#	Chart	Type	Library	Section	Purpose
N	Price start→end	Slope chart	matplotlib	6.6	Collapse time to a single comparison
O	AI-generated live chart	Stacked area	plotly (via AI code demo)	7.3	Prove the AI workflow works
P	Interactive pipeline demo	Interactive dashboard	Plotly Dash / Plotly + JS	8	Fulfill the "see pipeline animate" goal

Total: ~16 distinct chart types, deliberately diverse to serve the Data Viz course goal.

6. Library Cheat-Sheet Slide (for the deck appendix)

A single reference slide the presenter can flash during Q&A if asked "what libraries did you use?"

- **pandas** — the data-shape tool. `read_csv`, `merge`, `groupby`, `pivot_table`, `resample`.
- **numpy** — vectorized math.
- **matplotlib** — the workhorse. Full control, deliberately ugly by default so you have to make design decisions.
- **seaborn** — statistical charts done nicely. `heatmap`, `FacetGrid`, `boxplot`.
- **plotly** — interactive charts. Great for the web export.
- **holoviews / bokeh** — chord diagram, network-style flows.
- **scikit-learn** — `StandardScaler`, `KMeans`, `PCA`, silhouette scoring.
- **umap-learn** — better cluster-preserving dimensionality reduction.
- **scipy.stats** — KS test, other nonparametric tests.
- **geopandas + folium** — map layers (used in the extended 25-district version).
- **requests + tenacity** — API calls with retries for the raw data pulls.

7. Preprocessing Pipeline (Design Spec for the Jupyter Code)

The Jupyter notebook (to be built next) will implement this exact pipeline:

Stage 0 — Ingestion

- Pull OA-12252 (hourly) and OA-12914 (daily) via the Seoul Open Data Plaza API.
- Pull MOLIT apartment real-transactions for the 3 focus gu (Gangnam-gu, Mapo-gu for Hongdae, Yeongdeungpo-gu for Yeouido).
- Pull station coordinates and admin boundary GeoJSON.

- Persist raw pulls under `/data/raw/` with a fetch timestamp.

Stage 1 — Schema normalization

- Rename Korean columns to snake_case English.
- Enforce dtypes.
- Canonicalize station names (drop parenthetical alternates, unify punctuation).

Stage 2 — Entity resolution

- Join ridership tables to station coordinates via a `station_id` lookup.
- Assign each station to its administrative gu using a point-in-polygon check against the GeoJSON.

Stage 3 — Feature engineering per station

- Morning peak intensity (max ridership between 07:00–09:59, normalized to daily total)
- Evening peak intensity (max ridership between 17:00–19:59, normalized)
- Weekend/weekday ratio (mean weekend / mean weekday)
- Late-night share (share of taps between 22:00–01:59)
- Directional asymmetry at peak hour ($|\text{tap_in} - \text{tap_out}| / (\text{tap_in} + \text{tap_out})$)
- Coefficient of variation of hourly ridership across the year

Stage 4 — Standardization + clustering

- `StandardScaler` on the 6 features
- KMeans with k chosen by elbow + silhouette
- UMAP on standardized features for 2D visualization

Stage 5 — Price panel construction

- Compute a mean per-square-meter monthly price index per administrative gu
- Assign each station its host gu's price series
- Group by station cluster label → get per-cluster monthly price series

Stage 6 — Statistical tests

- KS test on hourly distributions for pairs of the 3 focus stations
- One-way ANOVA on price growth rates across clusters (with the standard caveats)

Stage 7 — Export figures

- Each of the ~16 charts is a callable function `make_chart_A(...)` etc, that returns a matplotlib Figure or a plotly Figure. Each callable writes a PNG (for the PPT) and an HTML (for the interactive site) to `/exports/`.

8. GitHub Pages Interactive Design Spec

The interactive page must let the audience click through the pipeline live during Section 8. Design:

Layout (desktop):

+-----+ Reading Seoul Through Its Movement – Interactive Demo +-----+			
RAW DATA	TRANSFORMATIONS	VISUALIZATION	
(left pane)	(middle strip)	(right pane)	
first 20	[pandas.groupby]	[live plotly chart]	
rows of	[resample('H')]		
OA-12252	[pivot_table]		
(scrollable)	[StandardScaler]		
	[KMeans.fit]		
	[UMAP]		
	each button has		
	a tooltip and		
	an arrow →		
+-----+			

Interaction model:

- Buttons in the middle strip fire in a defined sequence. Clicking a button both (a) updates the left pane to show the transformed dataframe, and (b) morphs the right pane's chart with a smooth transition (Plotly `animate` frames or CSS transitions on SVG).
- Each button has an "Explain" tooltip that in one sentence answers: *"Why is this step necessary for the final chart to be honest?"*

Tech stack (final validation to happen in deployment step):

- Option 1 (preferred per user): Plotly Dash → export to static HTML via `dash-static` or precompute all intermediate states and serve as vanilla HTML+JS. GitHub Pages is static-only, so a runtime Dash server cannot be hosted there; either an intermediate-state snapshot approach or a Dash deployment on a free-tier host (Render/Vercel) linked from GitHub Pages will be used.
- Option 2: Pure Plotly + vanilla JS. All computation done offline in the notebook, each state exported as JSON, JS on the page swaps traces on button click. This is lighter, deploys to raw GitHub Pages, has no server.

[NOTE: 사용자가 "Plotly Dash 스타일 풀 인터랙티브"를 원했는데, GitHub Pages는 정적 호스팅만 가능. 실제 Dash 서버가 필요하면 Render/Vercel 배포 후 GitHub Pages에서 링크 걸어야 함. Jupyter → Plotly HTML export 방식이 GitHub Pages 순수 배포에는 가장 안전. 배포 단계에서 최종 결정.]

9. Time-Budget Reality Check

Section	Planned	Cumulative
1 Hook	2:30	2:30
2 About me	3:00	5:30
3 Framing + hypotheses	3:00	8:30
4 Data journey	10:00	18:30
5 Testing H1	12:00	30:30
6 Deeper patterns	10:00	40:30
7 AI co-pilot	5:00	45:30
8 Interactive demo	3:00	48:30
9 Wrap-up	1:30	50:00
Q&A	10:00	60:00

Two contingency levers:

- If running long: skip Chart L (chord diagram) — it's the most cuttable.
- If running short: expand Chart F (KS/ECDF) with a second example pair, or add a supplementary slide contrasting a "bad chart" version of Chart A (e.g., three overlaid heatmaps in different color scales) to teach why the design choices in Chart A matter.

10. Next Deliverables (Downstream of This Document)

Once this design is approved, produce in this order:

1. `data_pipeline.ipynb` — Jupyter notebook implementing Sections 7's pipeline exactly, exporting all ~16 chart PNGs and HTMLs. Ready for Google Colab.
2. `chart_functions.py` — the reusable chart functions extracted for the interactive site.

3. `interactive/` — the GitHub Pages deployment (HTML + JS + exported Plotly JSON, or Dash app if hosted separately).
 4. **PPT deck** — only after all of the above is validated, when the user gives the go-ahead.
-

11. Notion / GitHub Portfolio Organization (Deliverable-Adjacent)

Per the user's ongoing portfolio-website strategy, this project will land in the `2026-Summer-Study` repo as a new dossier case file. Suggested placement:

- **Repo path:** `/projects/seoul-movement-lecture/`
 - `PRESENTATION_DESIGN.md` (this file)
 - `data_pipeline.ipynb`
 - `chart_functions.py`
 - `interactive/`
 - `exports/` (PNGs and HTMLs, gitignored except a curated subset)
 - `slides/` (PPT and PDF export)
 - `README.md` (public-facing project summary linking to the interactive site and the slide deck)
 - `projects-library.js` **entry:** new FILE No. under the DA/BI category, status SHIPPED after the talk, with the guest-lecture credential tagged. Featured Case eligible.
 - **Notion mirror:** an entry in the study tracker with links back to the repo and the deployed interactive URL, plus a private post-lecture retrospective note.
-

12. Fact-Check Log (Verified Before This Document Was Finalized)

- Seoul Open Data Plaza dataset **OA-12252 (hourly)** and **OA-12914 (daily)** subway ridership are confirmed live and current (verified July 2026). Update cadence: monthly for OA-12252, daily for OA-12914.
 - Seoul생활이동 (lifelog OD) dataset is under service renovation and NOT reliably usable at the moment; this document explicitly does not rely on it and instead reconstructs approximate flow from the tap-in/tap-out data with a limitation disclosure (Section 6.5).
 - MOLIT real-transaction API and station-coordinate open data are stable and standard for this kind of research.
 - Guest-lecture format confirmed by professor: 50-minute talk + 10-minute Q&A, MSc audience with weak coding/finance background, emphasis on graphs and code-behind-graphs.
-

End of design document. Ready for approval → then proceed to Jupyter code build.